Typed assembly for worst-case execution time analysis

Marcus Janum

May 15, 2026

r	::=		*registers:*	ι	::=	*instructions:*
		`r1` | $\cdots$ | `rk`				r_d`:=`v
v	::=		*operands:*			r_d`:=`$r_s + v$
		n	*integer literal*			`if` r`jump v`v
		ℓ	*label or pointer*	I	::=	*instruction sequences:*
		r	*registers*			`jump` v
						$\iota; I$

Figure 1: instructions and operands for the `tal0` language

1 Introduction

2 Typed Assembly Languages

Typed assembly languages are a group of languages (not unlike assembly) or rather just an idea that in varying degrees introduce type safety to programs written or compiled to assembly. The primary motivation for typed assembly languages is type safety and is strongly related to proof-carrying code [4, ch. 5].

In this section we will cover the canonical reason for introducing types as well as exploring levels of typing and implementations. Most of this section is based on Greg Morrissetts work whom has contributed most of the published research in this topic.

2.1 Why type safety in assembly?

Running untrusted code

2.2 TAL-0

As with PCC, it is of great interest to prove correctness of code. In modern programming languages this is done in part with a strongly typed syntax and semantics as "well-typed programs cannot 'go wrong'", thus moving some of the burden of correctness to the type system. The goal of TAL-0 (besides being a paedological tool from Morrissett to introduce readers to typed assembly) is to restrict control-flow to ensure control-flow safety of assembly code.

An example of a potential mishap — and how it may look solved — may be as the following risc-v snippet; jumping to the contents of a register, *hopefully* an address.

```
jump:
        jr      a0
```

This short example showcases how naive the machine is, as no checks need be performed, and no assurance is made, that 'a0' does indeed contain a valid code address. `tal0` solves this by special syntax and an abstract machine that only allows some evaluation; the above assembly snippet would be disallowed by the evaluation rules.

By figure 1, the previous code snippet would look like

```
jump:
    jump        r1
```

2.2.1 The abstract machine

The abstract machine presented here is in accordance to Morrissets TAL-0. The machine presented here will be extended in future sections.

2.2.2 Typing rules

$ftv(\tau)$ is the set of free variables for τ. Not sure where it looks to determine free-ness? a variable x is bound in τ if $\forall x.\tau$, but i think this may be a writing mistake? Surely it should be like $ftv(\forall x.y) = \{y\}$ while $ftv(\forall x.x) = \{\emptyset\}$

2.2.3 Soundness

The proof relies on the abstract machine not getting stuck.

3 worst-case execution time analysis

Worst-case execution time analysis is an important step in the process of developing real-time systems [5] like ssd_os. In general, and is easily examplified, not all programs are subject to execution time analysis, nor do all programs exit. The former is a consequence of the halting problem [7, ch. 13] [5].

3.1 Observed observation time

end-to-end measurements

3.2 static analysis

4 ssd_os (The system)

We are not really considering a heap at all (even though obviously program text exists, and `.words` can exist there, too).

4.1 Model

This project specifically targets the cacheless `v80` platform [1]. It consists of four clusters with four AMD MicroBlaze V processors, each having its own scratchpad memory, and all clusters connected to a shared scratchpad, referred to as shared moving forwards. The key take-away is that shared memory is slower than scratch memory.

4.2 Allocating

```
    struct memory_region
    {
        uint8_t *start;
        size_t   size;
    };

    struct arena_allocator
    {
        struct memory_region region;
        size_t               allocated;
    };
```

Figure 2: Structure of an allocator

```
uint8_t *arena_alloc(struct arena_allocator *alloc, size_t size, size_t
    alignment)
{
    uint8_t     *free_region_start = alloc->region.start + alloc->allocated;
    const size_t alignment_padding =
            (alignment - (((size_t) free_region_start) % alignment)) %
            alignment;
    const size_t size_to_alloc = size + alignment_padding;

    const size_t remaining =
            (alloc->region.start + alloc->region.size) - free_region_start;

    if (remaining < size_to_alloc)
        return NULL;

    uint8_t *allocated = free_region_start + alignment_padding;
    alloc->allocated += size_to_alloc;
    return allocated;
}
```

Figure 3: Example of an allocator function

5 Possibilities

5.1 Control flow

One of the motivations behind typed assembly languages as presented earlier (TODO: REF SECTION) is control flow. TAL-0 introduces types to code to restrict where jumps are allowed to target. Control-flow in regards to execution time analysis is not so interested in the validity of jumps but on the amount of jumps, i.e. loops.

Control-flow graph

By using DTAL and considering a program an oriented graph where blocks are nodes and branches are edges to create something like a control-flow graph.

5.2 Typing memory

The chosen platform is explicit in its memory hierarchy, which enables it to be explicit in code as well. This can be done through a type system, so any array or stack type is wrapped inside a memory region context.

5.3

6 Getting to a language

This section describes the process, the steps, and the considerations for designing the risc-v styled typed assembly language suited for static worst-case execution time analysis. Only a subset of the idea of the language will be fully fleshed out, and an even smaller part implemented. The "full" language requires much too complex typing that involves constraint satisfaction of predicate logic, far out of the scope of this project, unfortunately. "The language" refers to the implementation TAL designed for WCET. The language syntax is pictured in figure 4. All registers map to τ, except when otherwise noted. The register names follow the risc-v ABI names [3]. If a register is not explicitly typed, it will default to `top`. The jump instruction `j` is kept only to link blocks together, mimicking assembly fall-through.

6.1 Strays from risc-v

As with the TALs introduced above, there is little reason to distinguish between immediate instructions and register instructions, meaning that `subi` and `sub` can be treated the same. This is due to the value type that they all share in some form, defined in some way as

$$\text{value} \quad v \quad ::= \quad c \mid r$$

For code generation, these will have to be reintroduced, but this is trivial, since the value field will either be an immediate value or a register.

Since all types are word sized, we replace the load and store instructions with `load` r_d, $imm(r_s)$ and `store` r_s, $imm(r_d)$

6.2 Types

While the syntax of the language is inspired by DTAL, it is much simplified (or mangled, even). This is also the case of the τ types, where only a, `top`, and *int* remain. The array and stack types

type variables	α		
Register files	R	::=	$[\texttt{zero} : int, \texttt{ra} : \tau_1, \texttt{sp} : S, \cdots]$
region memory types variables	μ		
region memory types	M	::=	$scratch \mid shared \mid \mu$
stack types variables	ρ		
stack types	S	::=	$[] \mid \rho \mid \tau :: S$
Collection types	C	::=	$M\ \tau\ array(i) \mid M\ S$
types	τ	::=	$\alpha \mid \texttt{top} \mid int \mid C$
type variable contexts	Δ	::=	$\cdot_{\text{tv}} \mid \Delta, \mu \mid \Delta, \rho \mid \Delta, \alpha$
constants	c	::=	$i \mid l$
labels	l		
instructions	ins	::=	$aop\ r_d,\ r_s,\ v$
		|	$\texttt{incr}\ r_d,\ r_s,\ v$
		|	$\texttt{decr}\ r_d,\ r_s,\ v$
		|	$\texttt{mv}\ r_d,\ r_s$
arithmetic operations	aop	::=	$\texttt{add} \mid \texttt{sub} \mid \texttt{mul} \mid \texttt{div}$
instruction sequences	I	::=	$\texttt{j}\ v \mid \texttt{halt} \mid ins;\ I$
Blocks	B	::=	$\lambda\Delta.(R, I)$
Programs	P	::=	$l_1 : B_1; \cdots ; l_n : B_n$

Figure 4: Syntax for a typed assembly language.

are still present but wrapped, but collections will be explained later. Universally quantified types, i.e. type variables are still present, although they do not serve a concrete function. Stack type variables are still present. They are needed to append to a stack, which is needed for `incr`. Specific for the language, region memory type variables have been introduced. They allow labels to be polymorphic over regions. This is an ergonomic must, as otherwise the programmer (or a compiler) would have to write labels for both `scratch` and `shared`.

For the register file, this also means that any type that is not explicitly typed defaults to `top`. This means that for later blocks, `top` can freely be coerced into a new type.

6.3 Collections

This section describes how the array and stack types work with each other, and how they are to be thought of conceptually in the language moving forwards. Some TALs introduce `malloc` or similar pseudo-instructions to introduce a notion of types to arrays. Since assembly does not in of itself contain arrays, but only pointers, some level of abstraction needs to be introduced for a type system to be able to type arrays and differentiate them from garbage and labels. In `C`, heap arrays are pointers, and pointer arithmetic is allowed. Labels can only be used in `goto` statements, and not even arithmetic is permitted on them. Still, it is not trivial whether `float* n` points to a scalar, or is an array.

A `malloc` instruction is not suited in this case, since the target system contains multiple allocators (as discussed in section TODO: REF HERE, THE SYSTEM) working over memory regions. The problem then evolves into the question of how to design a type system that makes an allocator implicitly emit an array type, or a type that can be coerced into arrays. A simple allocator is shown in figure 5. It has no safety checks, and it does not align allocated memory. The TAL type systems often only accept word sized types, and for simplicity, so will I. To answer this, we first look into stacks, arrays, and a type system. Another issue is the fact that the stack

```
unsigned int *simple_malloc(struct arena_allocator *alloc, size_t size)
{
    unsigned int* start = alloc->region.start + alloc->allocated;
    alloc->allocated += size;
    return start;
}
```

Figure 5: A simple allocation function

pointer grows downwards, while the arena stacks grow upwards. This will also be solved.

6.3.1 Stacks

Stack types can be defined recursively as

$$\begin{array}{lll} \text{stack types variables} & \rho & \\ \text{stack types} & S & ::= \quad [] \mid \rho \mid \tau :: S \end{array}$$

[6]

In DTAL, the stack type is a part of the type state Σ, and is modified through `pushd` and `pop` operations, a very conventional way to use the stack. Here, stacks are used not only for the call-stack, but general allocations, and to be coerced into array types. To communicate to the type system that we intend to modify a stack, we introduce two instructions `incr` and `decr`. They may be described through the code generation in 8. `incr` decreases the stack pointer in r_s by v words (removing the need for runtime alignment) and places the value in r_d whose types will be that of r_s with v `top` values appended to it, e.g.,

```
main:   ('r)
        [sp: 'r]
        incr    a0, sp, 4   ; a0 : top :: top :: top :: top :: 'r
```

where `'r` is a stack type variable. `decr` functions as expected, popping v words off the stack.

A programmer may not be interested in passing on *the whole* stack after increasing the word count, such as stack arrays in C. If the type of a stack allocated array was the whole call-stack, not only would it obfuscate the intend, it would prevent memory safety (if the type system enforced such things). It is therefore desirable to "cut down" or slice a stack. This can be done by considering a stack S a subtype of a stack S' if S is contained within S' with a possibly empty tail. Or formally

$$\tau_0 :: \cdots :: \tau_n \leq \tau_0 :: \cdots :: \tau_m \mid n \leq m$$

To illustrate the usefulness, we can expand on the previous code snippet:

```
main:   ('r)
        [sp: 'r]
        incr    a0, sp, 4   ; a0 : top :: top :: top :: top :: 'r
subtype:
        [a0: top :: top :: top :: top]
        ..
```

While `top` is not so useful in of itself since it practically refers to garbage values, we will later introduce a subtyping rule to `top`, specifically such that any type can be coerced from `top`. This makes it possible to instantiate a stack of any type (although the language may not

```
start:  [a0: int, a1: int, a2: int, a3: int]
alloc:  [a0: int, a1: int, a2: int, a3: int]
        incr    a4, sp, 4       ; a4: top :: top :: top :: top :: ..
store:  ('m)
        [a0: int, a1: int, a2: int, a3: int, a4: 'm int array(4)]
        store   a0, 0(a4)
        store   a1, 1(a4)
        store   a2, 2(a4)
        store   a3, 3(a4)
```

Figure 6: Example code for allocating a stack array and writing to it

support so many types) without introducing special "typed" syntax to `incr` and `decr`, as DTALs *newarray*[τ]. So really we could write:

```
main:   ('r)
        [sp: 'r]
        incr    a0, sp, 4   ; a0 : top :: top :: top :: top :: 'r
subtype:('a)
        [a0: 'a :: 'a :: 'a :: 'a]
        ..
```

for some type variable `'a`.

6.3.2 Arrays

Since we have foregone array initialization instructions, we add the choice to coerce a stack containing only one type to an array of that type. This models C stack arrays quite well.

6.3.3 Array allocation using stacks

At this point we can subtype stacks to create smaller stacks, and we can instantiate array types by coercing a stack.

While the call-stack grows downwards and `incr` and `decr` can be used as expected, the memory region allocations grows upwards. The solution lies in the conceptualization.

While `incr` and `decr` only work on immediate values here, this is only for simplification, and since the language contains no looping constructs, there is no value in having dependent sizes.

An example program that allocates a stack array and stores four integers could look like the program in figure 6.

`::` is right associative, so `scratch top :: top ..` = `scratch`(`top :: top ..`)

`top` can be coerced into any type, and is used to initialize the array.

A stack [τ_0`::`$\cdots$`::`τ_n] can be coerced into an array τ *array*(`m`) iff $m < n$ and $\tau = tau_i$ for all $0 \leq i < m$

`'m` is a type variable that will be unified to type `scratch`.

Since the stack pointer points to the stack, and the target is a specific ssd os system (link) the type of `sp` will always be `scratch`S.

Another use is working with the arena allocators. They are simple, and also work like a stack, but these grow upwards, which is opposite the call stack and therefore the typing rules would not work.

$$
\begin{array}{lcl}
\Delta_0 & = & \cdot_{\mathrm{tv}} \\
\Delta_1 & = & \texttt{'sp} \\
\Delta_2 & = & \texttt{'m} \\
R_0 & = & \texttt{a0} : \mathit{int}, \texttt{a1} : \mathit{int}, \texttt{a2} : \mathit{int}, \texttt{a3} : \mathit{int}, \\
R_1 & = & \texttt{a0} : \mathit{int}, \texttt{a1} : \mathit{int}, \texttt{a2} : \mathit{int}, \texttt{a3} : \mathit{int}, \texttt{sp} : \texttt{'sp} \\
R_2 & = & \texttt{a0} : \mathit{int}, \texttt{a1} : \mathit{int}, \texttt{a2} : \mathit{int}, \texttt{a3} : \mathit{int}, \texttt{a4} : \texttt{'m}\ \mathit{int\ array}(4) \\
I_0 & = & \texttt{j alloc} \\
I_1 & = & \texttt{incr a4, sp, 4} \\
 & & \texttt{j store} \\
I_2 & = & \texttt{store a0, 0(a4)} \\
 & & \texttt{store a1, 1(a4)} \\
 & & \texttt{store a2, 2(a4)} \\
 & & \texttt{store a3, 3(a4)} \\
 & & \texttt{halt} \\
B_0 & = & \lambda\Delta_0.(R_0, I_0) \\
B_1 & = & \lambda\Delta_1.(R_1, I_1) \\
B_2 & = & \lambda\Delta_2.(R_2, I_2) \\
P & = & \texttt{start} : B_0;\ \texttt{alloc} : B_1;\ \texttt{store} : B_2
\end{array}
$$

Figure 7: Explanation of code in figure 6

```
struct memory_region
{
    uint8_t *start;
    size_t   size;
};

struct arena_allocator
{
    struct memory_region region;
    size_t               allocated;
};
```

While the purpose of the language is only typed memory regions,

6.3.4 Unitialized values

Since the stack `incr` instruction essentially creates garbage values, we may want to type it, and introduce a notion of instantiated and uninstantiated types [2].

Essentially, the regions type — while in C is just a pointer — is a lifted stack type containing the whole region. When memory is allocated in an arena, the type of the returned memory is a coerced slice of the arena. If an arena has region of type $\texttt{top}_0\texttt{::}\cdots\texttt{::top}_n\texttt{::[]}$, then an offset of k words gives the stack $\texttt{top}_k\texttt{::}\cdots\texttt{::top}_n\texttt{::[]}$, which can be coerced into an array of fitting type and size.

Since the language — like other TALs — only work on word sized types but operations on memory in risc-v (like moving the stack pointer) is not aligned, we want to constrain the syntax to only allow word-size operations. Only stacks can change size (through arithmetic operations on a stack pointer) so it suffices to introduce instructions `incr` and `decr`. They may be best explained through their code generation, shown in figure 8

$$
\texttt{incr}\ r_d,\ r_s,\ v \quad \mapsto \quad \begin{cases} \texttt{mul}, \quad r_d, \quad v, \quad \mathit{wordsize} \\ \texttt{sub}, \quad r_d, \quad r_s, \quad \mathit{rd} \end{cases} \quad \text{iff} \quad \Phi;\Delta;R \vdash v : \text{is register}
$$

$$
\qquad\qquad\qquad\quad \texttt{subi}, \quad r_d, \quad r_s, \quad v \cdot \mathit{wordsize} \quad \text{iff} \quad \Phi;\Delta;R \vdash v : \text{is immediate value}
$$

$$
\texttt{decr}\ r_d,\ r_s,\ v \quad \mapsto \quad \begin{cases} \texttt{mul}, \quad r_d, \quad v, \quad \mathit{wordsize} \\ \texttt{add}, \quad r_d, \quad r_s, \quad \mathit{rd} \end{cases} \quad \text{iff} \quad \Phi;\Delta;R \vdash v : \text{is register}
$$

$$
\qquad\qquad\qquad\quad \texttt{addi}, \quad r_d, \quad r_s, \quad v \cdot \mathit{wordsize} \quad \text{iff} \quad \Phi;\Delta;R \vdash v : \text{is immediate value}
$$

Figure 8: Code generation for `incr` and `decr`

6.4 Abstract machine

Unlike other TAL abstract machines, I am omitting the stack, since it will be used as a normal register (`sp`). The DTAL abstract machine has a machine state $\mathcal{M}(\mathcal{H},\mathcal{R},\mathcal{S})$ being a triple over a heap, register file, and stack. Since the system the language is targeted at is stack based, no heap is needed (except for program text, but this is omitted partially due to program scope) and no explicit stack is needed, since the register `sp` will be used. A such, the abstract machine contains only a finite mapping between registers and values.

This is also the motivation behind removing the DTAL type state Σ and state type σ.

6.5 Dynamic Semantics

6.6 Dependent types

Because of the restrictions on the implementation domain, we can restrict the dependent types to contain fewer index propositions.

```
L0:     {n: int | n > 0}
        [a0: int(n)]
        muli    a1, a0, 4       ; a1: n * 4
        sub     sp, sp, a1      ; sp: top_0 :: ... :: top_n :: sp
        mv      a1, a0
        mv      a0, sp
L1:     ('m)
        {n: int | n > 0}
        [a0: 'm int array(n), a1: int(n)]
```

Is a valid program. The two `n` s are locally scoped, so they share no information, however they should represent the same value in this case. Since `n` is not known, only that it is a natural non-zero number, it is not known how many words are pushed to the stack, therefore its type depends on `n`.

However, since the lack of control flow, loops do not exist and the language turns novel. Then there is no way to store `n` words, and therefore no reason to be able to allocate a variable amount of words. The language reduces to constant expressions. In future work, reintroducing control flow is a must, and as discussed previously (TODO: HOPEFULLY) dependent types can be used to infer some loop iterations. For this, we would sorely miss type variables in index expressions.

```
start:  [a0: int, a1: int, a2: int, a3: int]
        subi    sp, sp, 16      ; sp: scratch top :: top :: top :: top :: 'r
        mv      a4, sp
coerce: ('m)
        [a0: int, a1: int, a2: int, a3: int, a4: 'm int array(4)]
        lw      a0, 0(a4)       ; a4: scratch int :: ..
        lw      a1, 4(a4)       ; a4: scratch int :: int :: ..
        lw      a2, 8(a4)       ; a4: scratch int :: int :: int ..
        lw      a3, 12(a4)      ; a4: scratch int :: int :: int :: int :: 'r
```

Figure 9: Example program

this reduces stack type rules to

$$\frac{}{\Phi;\ \Delta;\ R \vdash \mathsf{sub}\ r_d,\ r_s,\ v}\ (\mathsf{sub})$$

6.7 Dynamic Semantics

6.8 Coercion

6.9 Syntax

The syntax for the language is described in figure 4, which takes inspiration from DTAL [6]. As have been discussed, in a complete language with control-flow, type variables should be reintroduced into index expressions.

An example program could look like figure 9

7 Implementation (wrong title)

7.1 Domain

Only a small subset of what has been discussed will be implemented, namely typing memory region types for the scratchpad system model. The typing rules will ensure that it is known before runtime of which memory region each load/store operation writes or reads from. Since the system works without cache (scratch region effectively taking its place) all IO has a set cost, and as do the arithmetic instructions. The problem then boils down to essentially summing up the cost of each instruction.

The solution is somewhat novel, since there is a painful lack of control-flow. In the theory description some outline of how this may be thought of and implemented is discussed. The biggest constraint and factor of deciding that control-flow is out of scope is due to the solving process (predicate logic SAT solving).

References

[1] Philippe Bonnet. σ: An operating system for solid-state drives.

[2] Greg Morrisett. stack-based typed assembly language.

[3] David A. Patterson and John L. Hennessy. Computer organization and design risc-v edition.

[4] Benjamin C. Price. Advanced topics in types and programming languages.

[5] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström. The worst-case execution-time problem—overview of methods and survey of tools. *ACM Trans. Embed. Comput. Syst.*, 7(3), May 2008.

[6] Hongwei Xi and Robert Harper. A dependently typed assembly language. *SIGPLAN Not.*, 36(10):169–180, October 2001.

[7] Torben Ægidius Mogensen. Programming language design and implementation.